

MMU: paginarea și memoria virtuală

5. Algoritmi de înlocuire:

Sunt necesari 2 algoritmi de înlocuire:

- Algoritmul de selecție și înlocuire a unei pagini MP (implementare mixtă *software-hardware*)
- Algoritmul de înlocuire a unei intrări TLB (implementat în *hardware*)

În funcție de monitorizarea/nemonitorizarea utilizării paginilor:

- algoritmi bazați pe gradul de utilizare a paginilor
- algoritmi care nu monitorizează gradul de utilizare a paginilor.

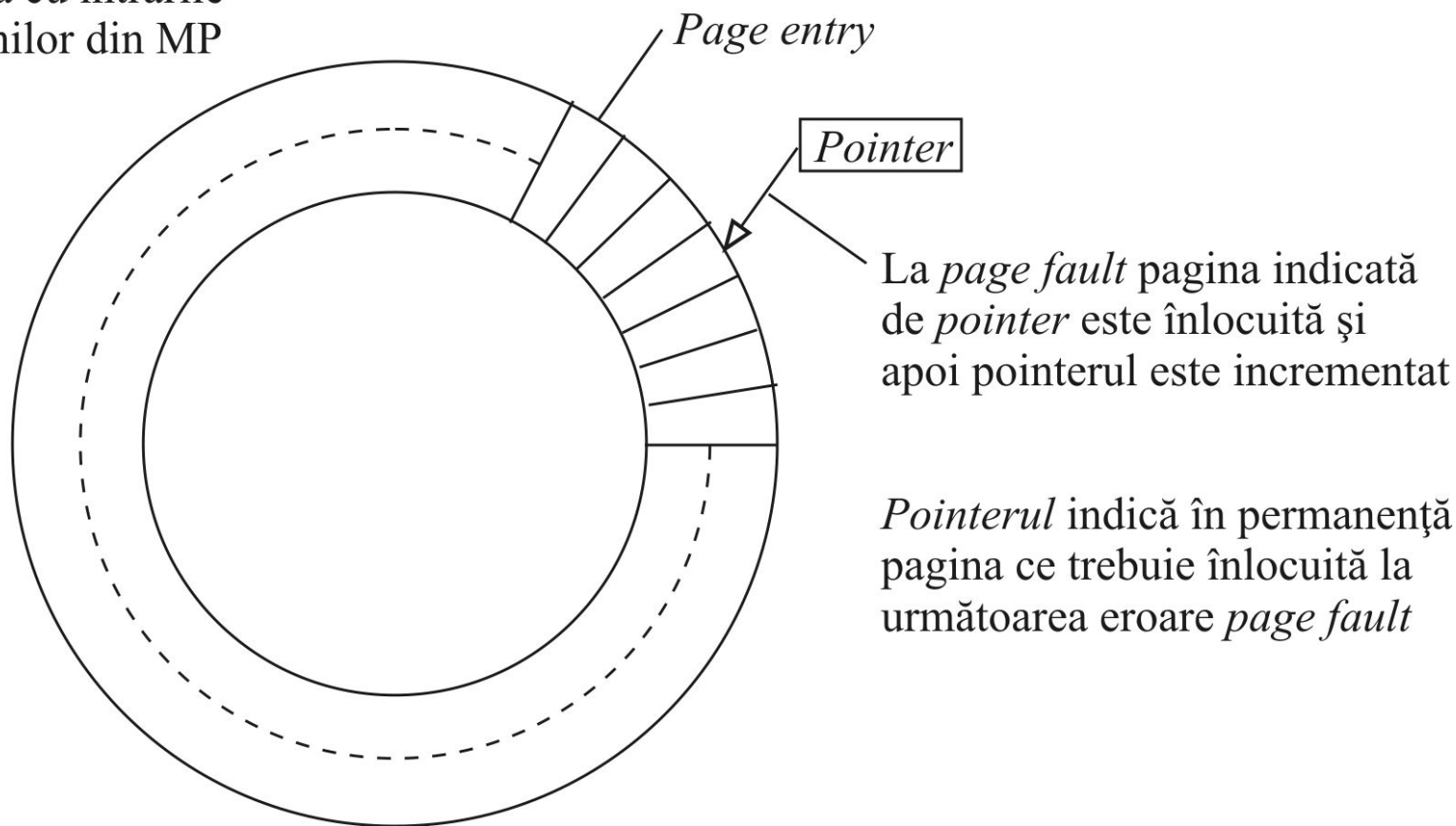
În funcție de modul de operare, algoritmi de înlocuire (bazați sau nebazați pe gradul de utilizare a paginilor) sunt de două tipuri:

- algoritmi globali
- algoritmi locali

MMU: paginarea și memoria virtuală

5. Algoritmi de înlocuire:

Listă circulară cu intrările
aferente paginilor din MP

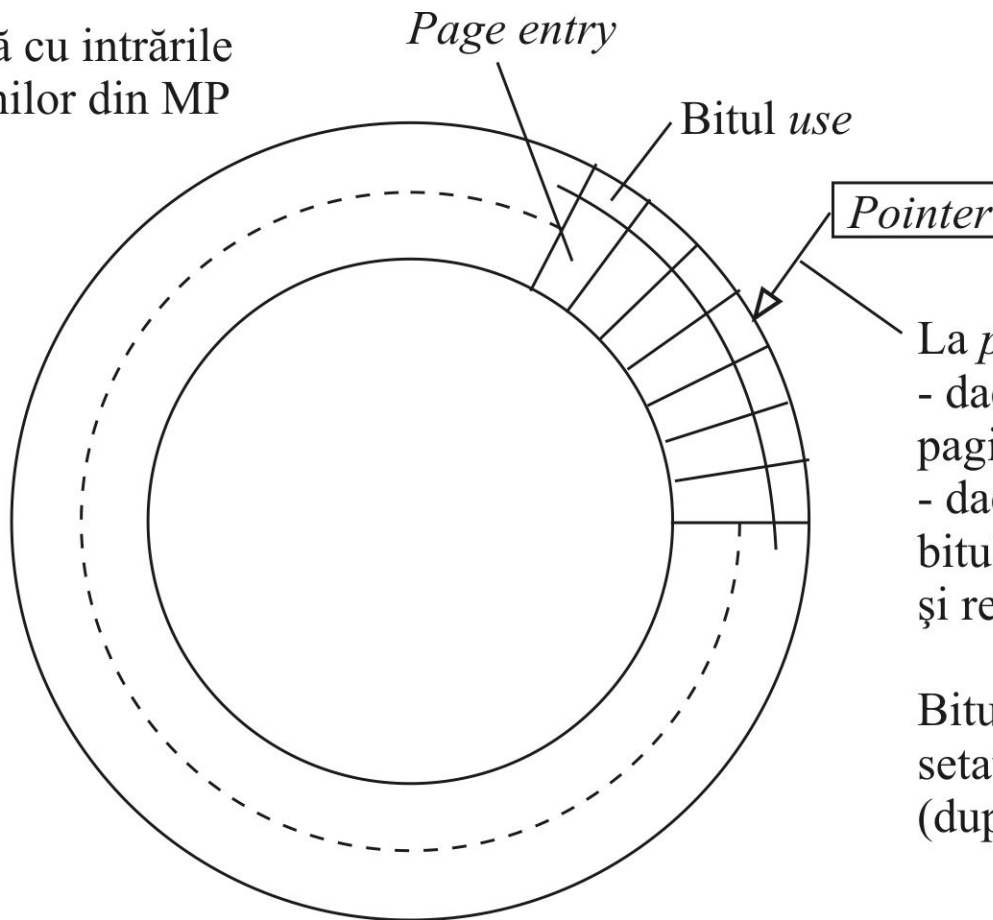


Algoritmul FIFO (*first-in first-out*) implementat cu o listă circulară

MMU: paginarea și memoria virtuală

5. Algoritmi de înlocuire:

Listă circulară cu intrările
aferente paginilor din MP



La *page fault*:

- dacă bitul *use* = 0, înlocuiește pagina și incrementează *pointerul*
- dacă bitul *use* = 1, resetează bitul, incrementează *pointerul* și repetă.

Bitul *use* aferent unei pagini este setat dacă pagina este referită (după înlocuirea inițială).

Algoritmul FINUFO (*first-in not-used first-out*) implementat cu o listă circulară

MMU: paginarea și memoria virtuală

5. Algoritmi de înlocuire:

Algoritmul **Least Recently Used (LRU)**:

Este cel mai frecvent utilizat în practică. Se implementează de regulă într-o variantă aproximativă, bazată pe biți **U-used** sau **A-accessed** (implementați în *hardware*) și contoare (implementate în *software*). Biții **U-used** sunt biți de control implementați în intrările în pagini (*page entries*).

La fiecare acces într-o pagină, bitul **U** din *page entry* este setat automat (*hardware*).

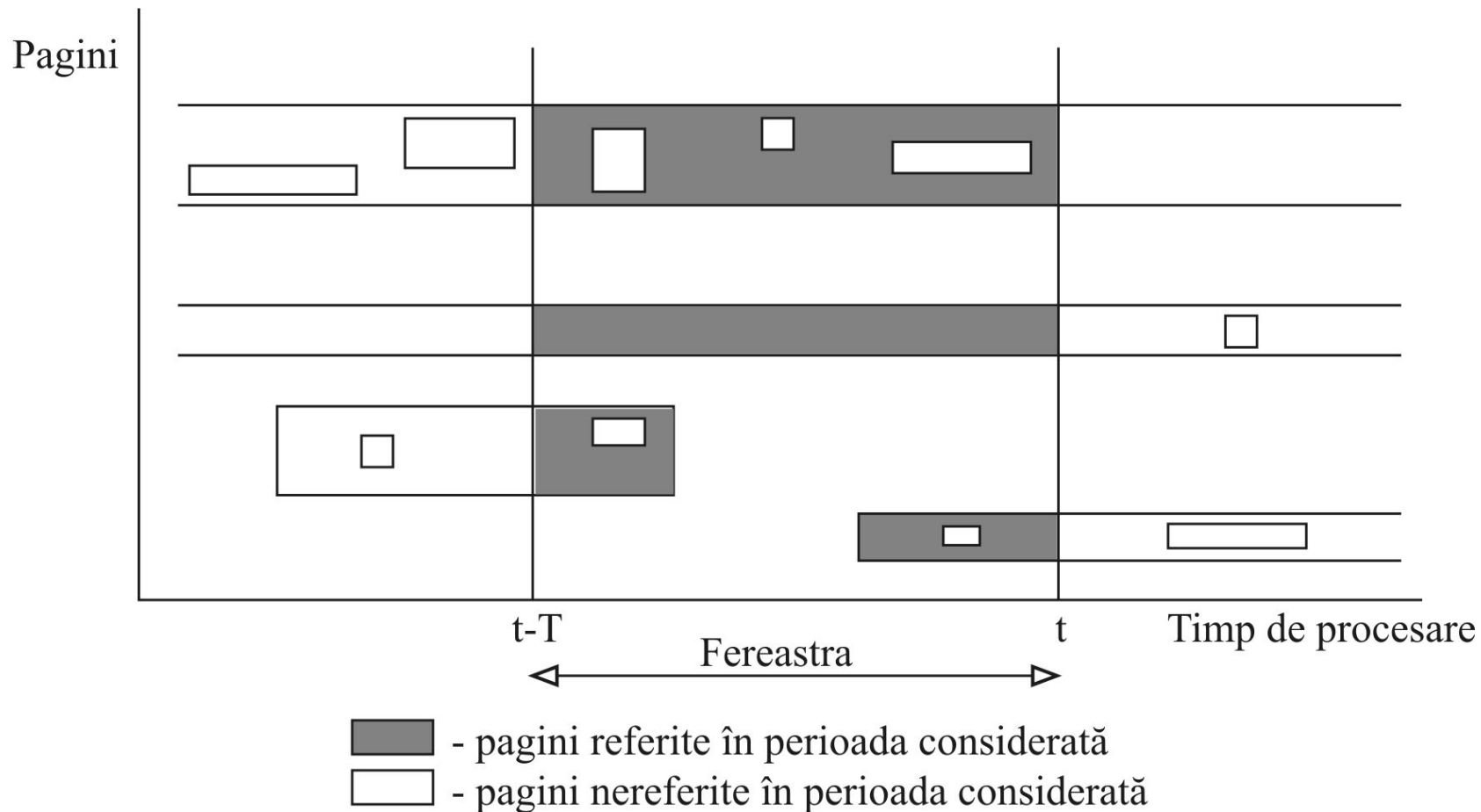
Pe de altă parte, biții **U** (din toate intrările) sunt scanați (citiți) în *handler*-ul de timp real. Dacă un bit **U** este găsit în 1, va fi resetat și de asemenea și contorul aferent este resetat (pagină utilizată pe ultimul interval). Dacă un bit **U** este găsit în 0, contorul aferent este incrementat (pagină neutilizată pe ultimul interval).

La *page fault*, se caută contorul cu valoarea maximă și pagina corespondentă este selectată pentru evacuare pe disc (este cea mai puțin recent utilizată).

De ce implementarea descrisă conduce la un algoritm **LRU aproximativ** ?

MMU: paginarea și memoria virtuală

5. Algoritmi de înlocuire:



Algoritmul **working set** (definirea setului de lucru cu ajutorul ferestrei de lățime T)

MMU: paginarea și memoria virtuală

6. Cost și performanțe:

Costul mediu per bit memorat, pentru un sistem de memorie ierarhizată care conține MP (RAM) și MS (disc):

$$C_{med} = \frac{c_m \cdot m_m + c_s \cdot m_s}{m_m + m_s}$$

unde c_m și c_s reprezintă costurile per bit pentru MP și respectiv MS iar m_m și m_s reprezintă capacitățile MP și respectiv MS.

Produsul spațiu-timp ($S \cdot T$) reprezintă produsul dintre memoria utilizată de un program și timpul de execuție al programului respectiv; este privit ca un indicator al costului execuției programului în cauză.

Partiții variabile: $S \cdot T(t_1, t_2) = \int_{t_1}^{t_2} M(t) \cdot dt$ - setul de pagini rezident în memorie la momentul t

Integrala devine o suma deoarece $M(t)$ se modifică numai la *page fault* !

n - numărul de referințe la memorie

Partiții fixe: $S \cdot T = M \cdot (n + D \cdot f)$ D - timpul mediu de transfer al unei pagini de pe disc în MP

f - numărul de erori *page fault*

MMU: paginarea și memoria virtuală

6. Cost și performanțe:

În particular partiția se modifică în momentele *page fault*:

$$S \cdot T = \sum_{t=1}^T M(t) + D \cdot \sum_{i=1}^f M(t_i) = M_{med} \cdot T + D \cdot \sum_{i=1}^f M(t_i)$$

unde: T – timpul total de execuție al programului

M_{med} – partiția (alocarea) medie pe durata execuției programului

t_i – timpul (momentul) în care se activează eroarea *page fault* numărul i .

Pentru a reduce costurile, produsul spațiu-timp trebuie minimizat.

Timpul de acces la memorie va fi dat de relația:

$$t_a = t_m + (1-h) \cdot t_s$$

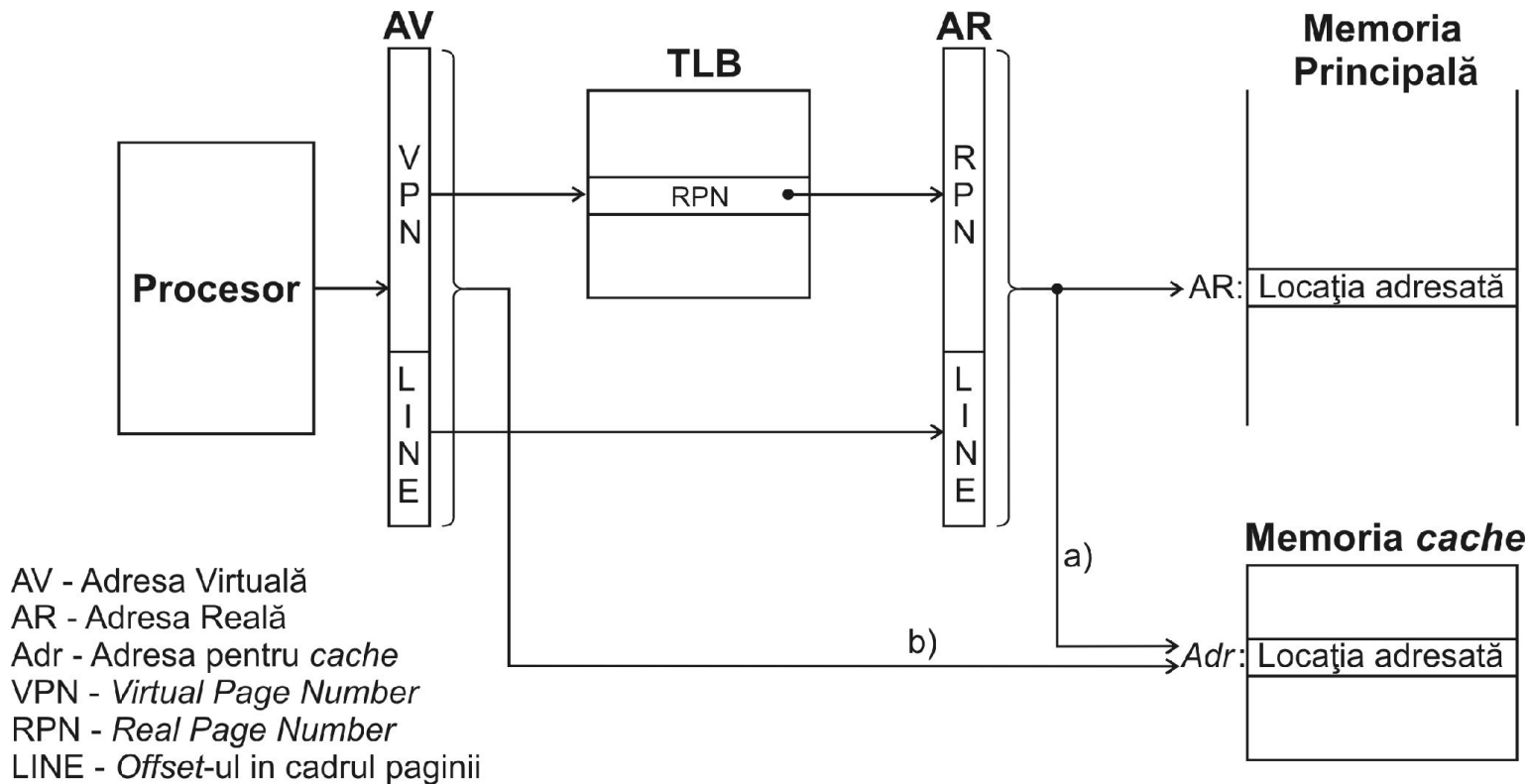
unde: t_m – timpul de acces la MP (inclusiv timpul necesar translatării adresei)

t_s – timpul adițional necesar accesării datelor pe disc

Timpul de translatare a adresei (t_t): $t_t = t_{tlb} + (1 - h_{tlb}) \cdot t_{mt}$

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:



Cele două variante de adresare a memoriei cache:

- a) cache adresat cu adrese reale
- b) cache adresat cu adrese virtuale

Timpul de acces la memorie în cazul acceselor cu hit în TLB și respectiv *cache*:

- a) $t_a = t_{TLB} + t_c$ (timp de traducere adresă prin TLB + timp de acces la *cache*)
- b) $t_a = t_c$ (timp de acces la *cache*)

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

A. *Cache* adresat cu adrese reale

Se observă că, timpul de acces la memorie în cazul acceselor cu *hit* în TLB și respectiv *cache* (cazul cel mai frecvent) este mai mic ($t_a = t_c$) în cazul *cache*-ului adresat cu adrese virtuale și semnificativ mai mare ($t_a = t_{TLB} + t_c$) în cazul *cache*-ului adresat cu adrese reale.

Rezultă: Din perspectiva performanțelor (timpului de acces), *cache*-ul adresat cu adrese virtuale este de preferat !

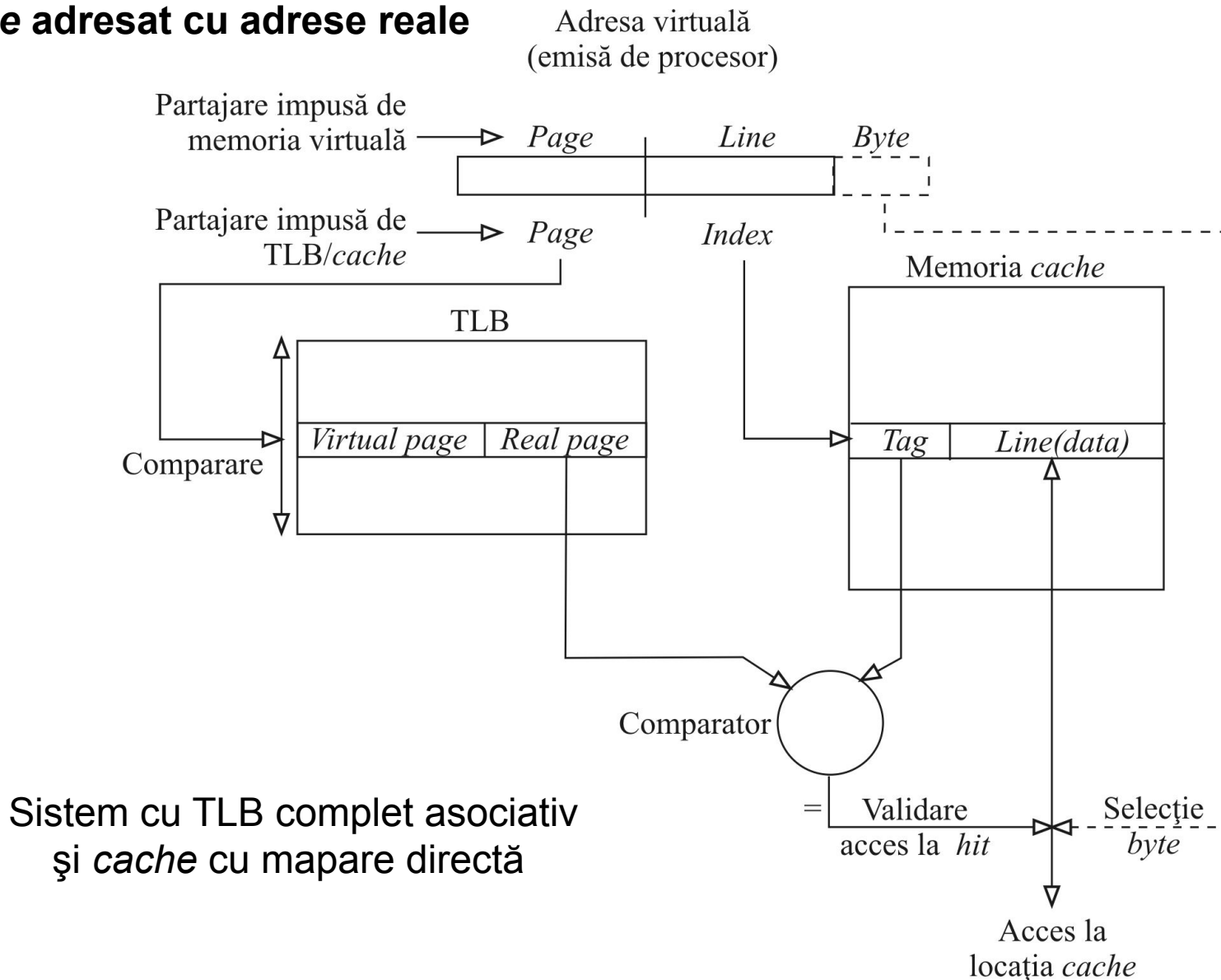
Întrebare: Poate fi redus timpul de acces la memorie în cazul *cache*-ului adresat cu adrese reale, până la limita $t_a = t_c$? Dacă da, în ce condiții ?

Răspuns: Da, timpul de acces la memorie, în cazul *cache*-ului adresat cu adrese reale, poate fi redus până la limita $t_a = t_c$, dacă numărul de biți din câmpul ***Index*** (impus de *cache*) este mai mic sau cel mult egal cu numărul de biți din câmpul ***Line*** (impus de memoria virtuală). Răspunsul poate fi argumentat pe baza celor trei scheme de principiu care urmează !

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

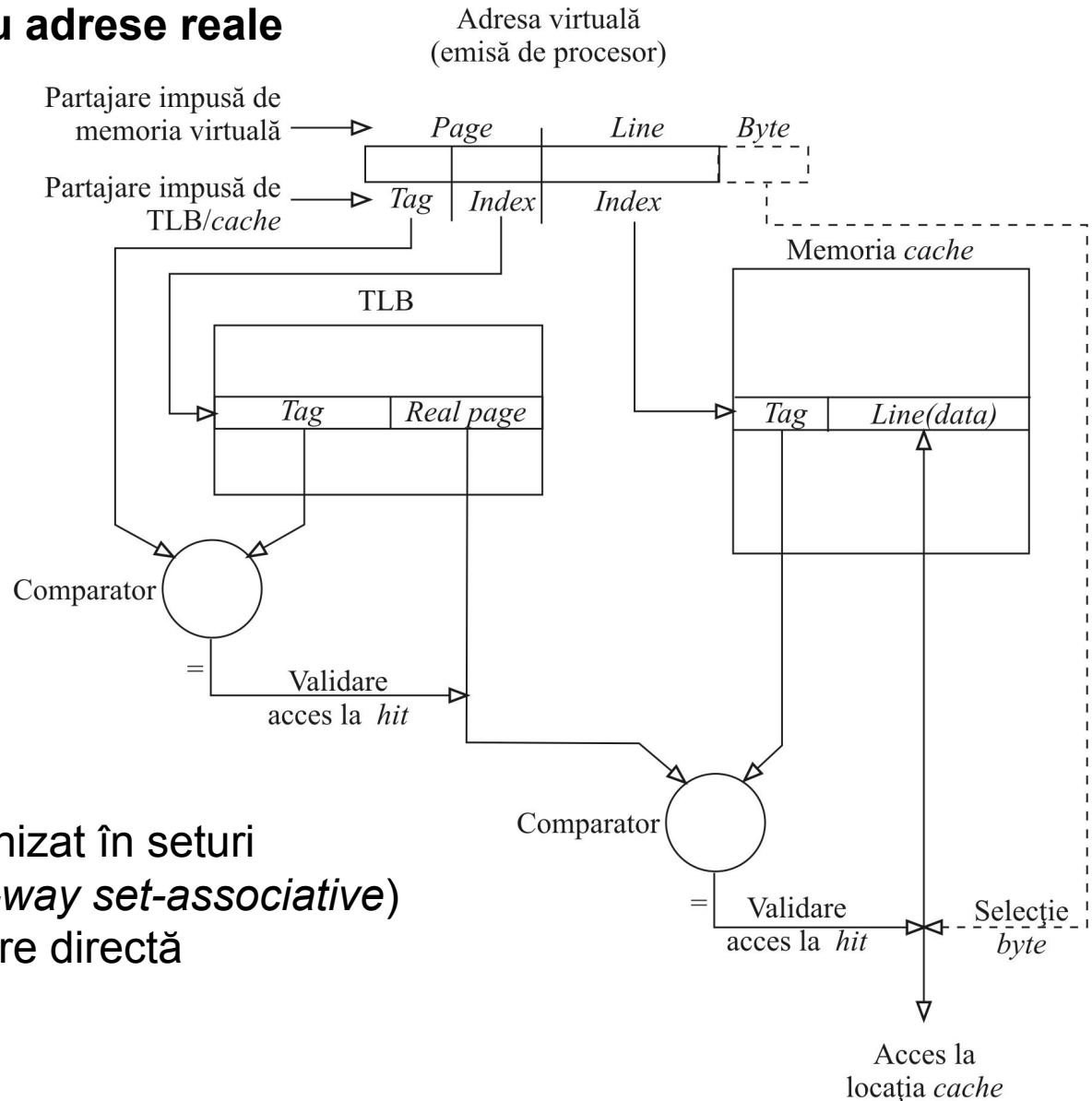
A. *Cache* adresat cu adrese reale



MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

A. *Cache* adresat cu adrese reale

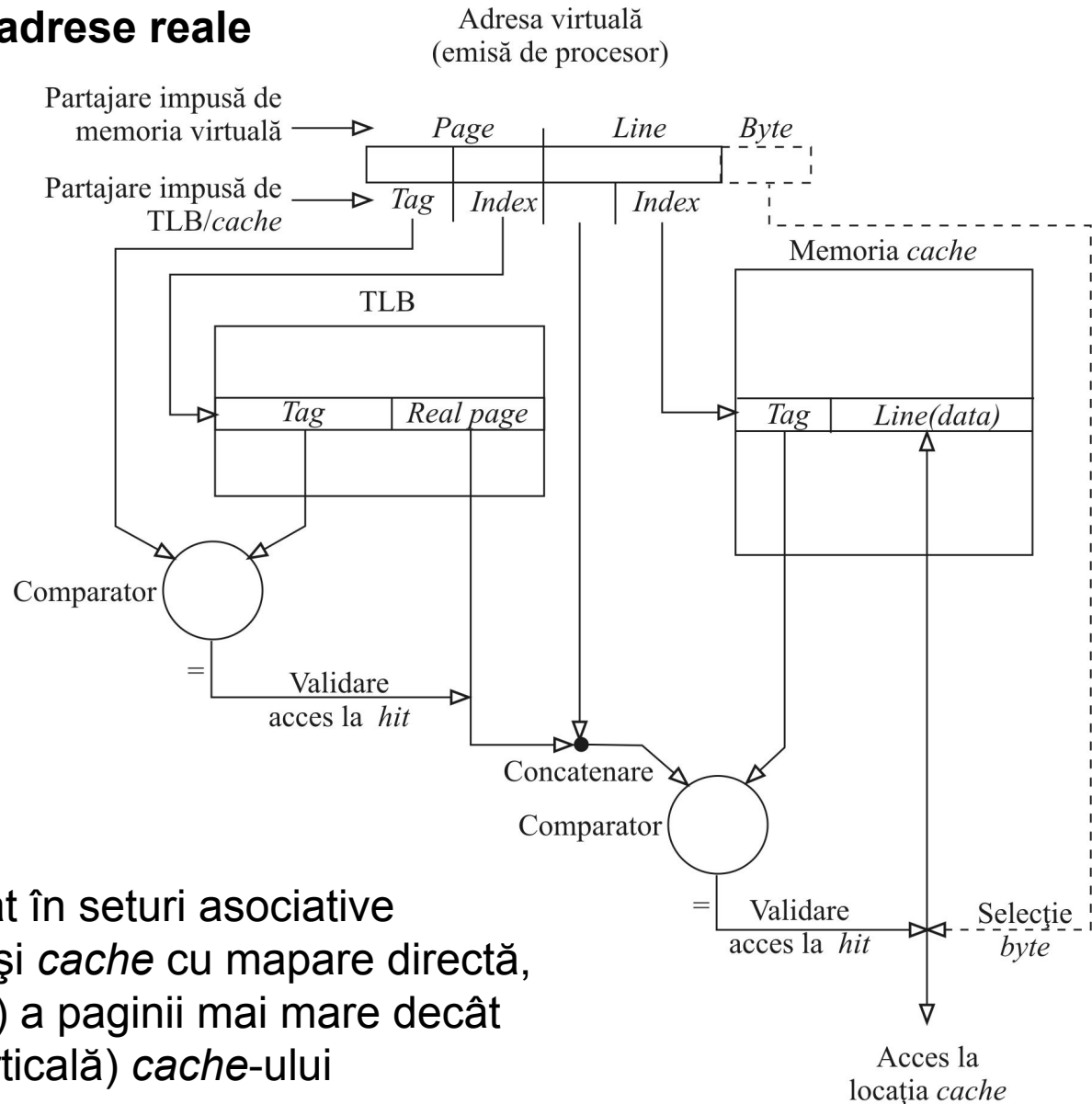


Sistem cu TLB organizat în seturi asociative (aici *one-way set-associative*) și *cache* cu mapare directă

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

A. *Cache* adresat cu adrese reale

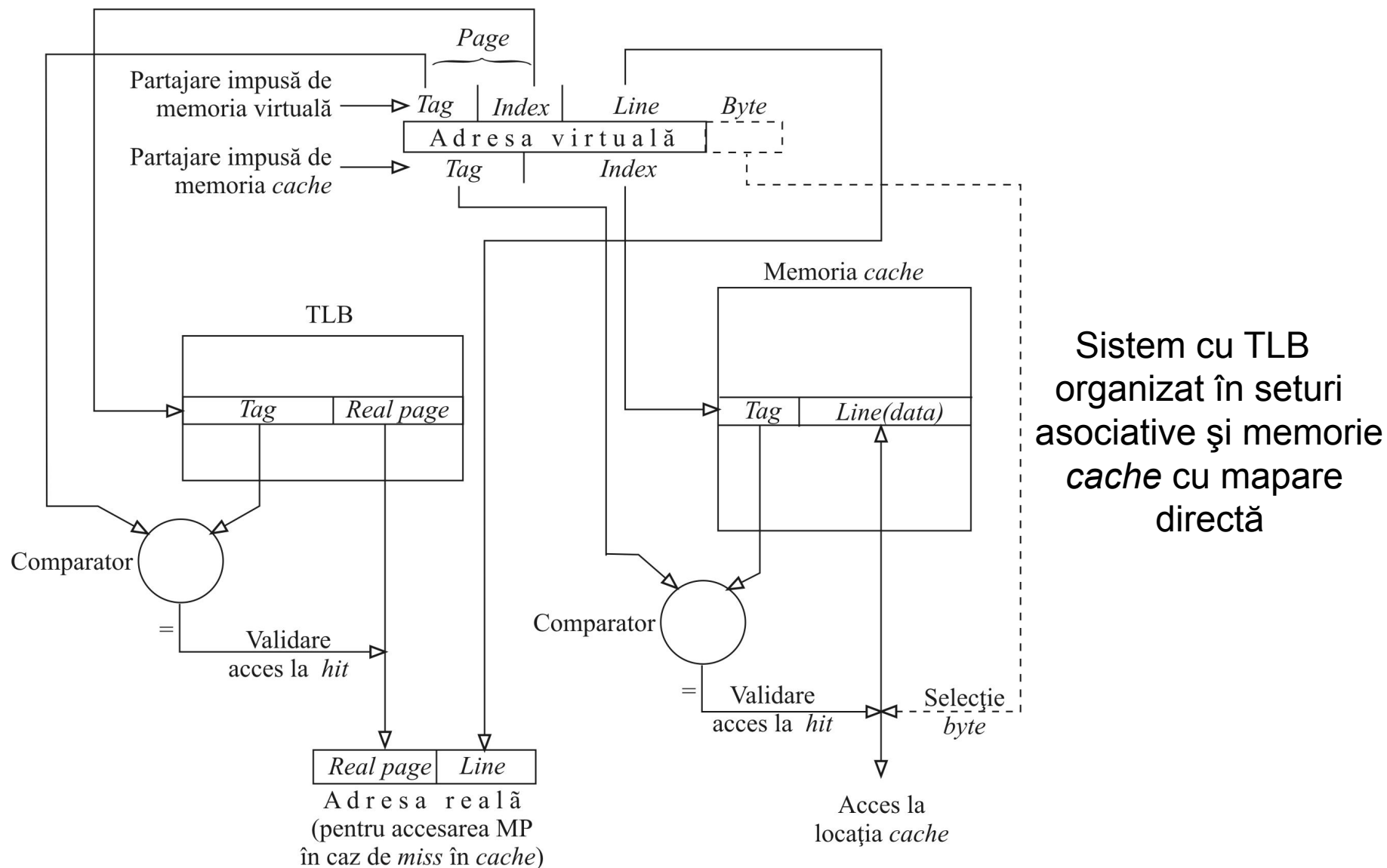


Sistem cu TLB organizat în seturi asociative (aici *one-way set-associative*) și *cache* cu mapare directă, cu dimensiunea (pe verticală) a paginii mai mare decât dimensiunea (pe verticală) *cache*-ului

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

B. *Cache* adresat cu adrese virtuale



MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

B. *Cache* adresat cu adrese virtuale

Avantaje:

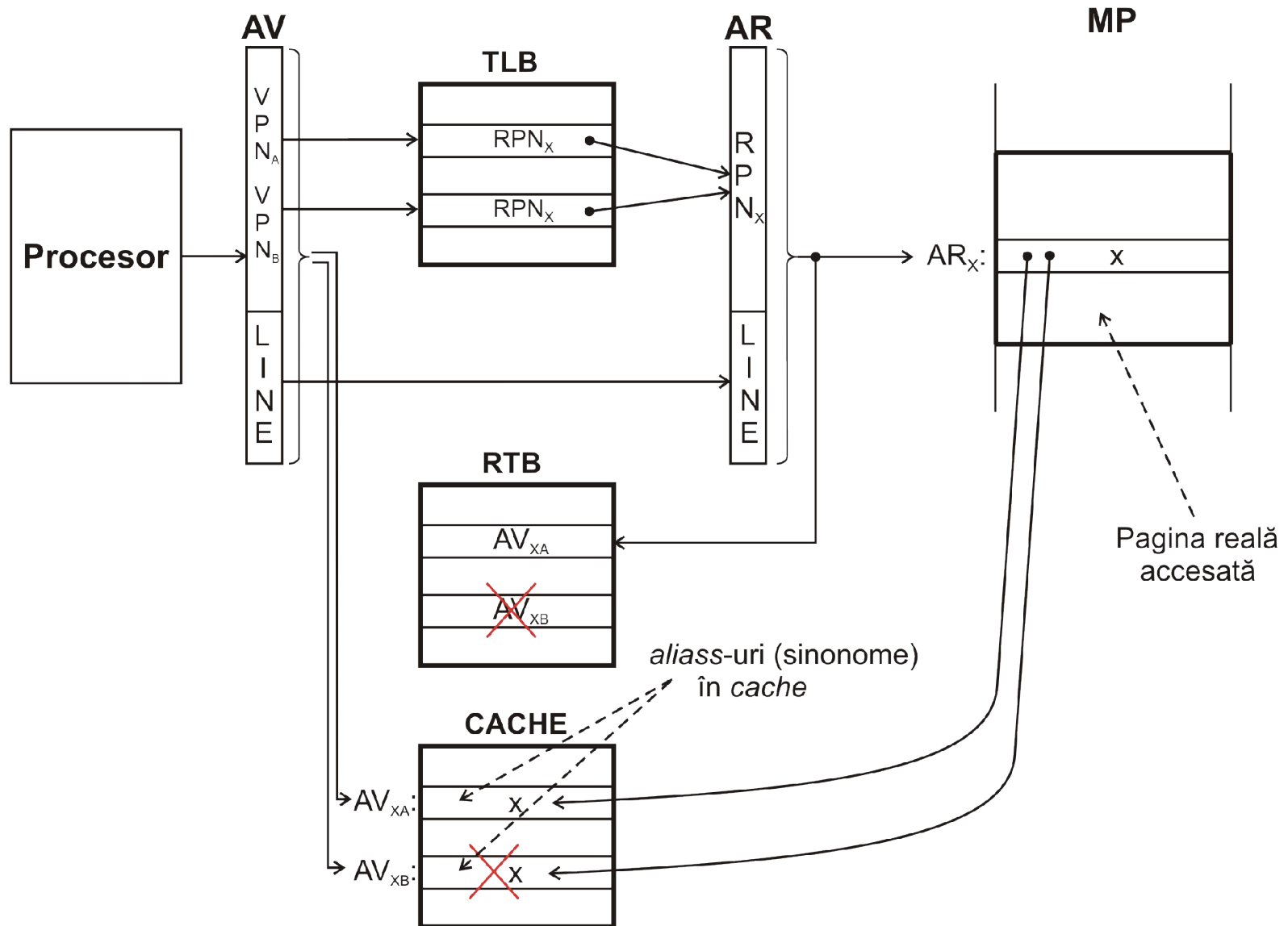
- în ciclurile de acces la memorie inițiate de procesor, adresele virtuale sunt imediat disponibile permițând creșterea vitezei de operare în raport cu *cache*-urile adresate cu adrese reale.
- divizarea adresei virtuale în câmpuri impusă de memoria virtuală nu mai are nicio legătură cu împărțirea adresei virtuale în câmpuri impusă de memoria *cache*; devin probleme de proiectare absolut independente.

Dezavantaje:

- complicații legate de gestiunea *alias*-urilor sau *sinonimelor* care pot să apară în *cache* (două adrese virtuale diferite, care aparțin la două procese diferite și care conduc prin translatare la aceeași adresă reală).
Asta înseamnă că în *cache*-ul adresat cu adrese virtuale pot să apară multiple copii (*alias*-uri) ale aceleiași locații din memoria principală (varibilă globală, partajată de două sau mai multe procese). Gestiunea *alias*-urilor prin *tag*-uri de proces sau *buffer*-e de translatare inversă (RTB-Reverse Translation Buffer sau ITB-Inverse Translation Buffer) !
- dacă DMA-urile accesează MP cu adrese reale, liniile *cache* asociate acestor adrese trebuie identificate (cu ajutorul RTB) și invalidate.

Apariția *alias*-urilor în *cache*-urile adresate cu adrese virtuale

Fie P_A și P_B două procese (funcții, clase) din cadrul unui program care accesează variabila globală X :
..- P_A accesează variabila X cu adresa virtuală AV_{xA} ; P_B accesează variabila X cu adresa virtuală AV_{xB}
- AV_{xA} diferă de AV_{xB} doar prin câmpul VPN (VPN_A și respectiv VPN_B), nu și prin câmpul LINE.



MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*: Timpul mediu de acces la memorie

Timpul mediu de acces la memorie aferent unui sistem care conține atât memorie *cache* (pe care o considerăm aici adresată cu adrese reale) cât și memorie virtuală paginată, are mai multe componente.

Derularea acceselor la memorie se poate face în 6 combinații posibile:

1. adresa în TLB, datele în *cache*
2. adresa în TLB, datele în MP
3. adresa în *cache*, datele în *cache*
4. adresa în *cache*, datele în MP
5. adresa în MP, datele în *cache*
6. adresa în MP, datele în MP

Vom considera *cache*-ul adresat cu adrese reale și vom presupune că nu există suprapunere între operația de traducere a adresei în TLB și cea de accesare a *cache*-ului.

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*:

Timpul mediu de acces la memorie:

Vom lua în considerare următoarele valori ale parametrilor care interesează:

- a) Timp de traducere adresă în TLB (sau de generare TLB-*miss*): $t_{TLB}=25$ ns
- b) Timp de interogare *cache* (timp *hit/miss* în *cache*): $t_{H/MC}=25$ ns
- c) Timp de citire (*fetch data*) din *cache*: $t_{FDC}=25$ ns
- d) Timp de acces la MP (citire din MP): $t_{MP}=200$ ns
- e) Rata de *hit* în TLB: $H_{TLB}=0,9$
- f) Rata de *hit* în *cache*: $H_C=0,95$

Timpii de acces și probabilitățile aferente celor 6 combinații sub care se poate

Comb.	Timp de acces	Probabilitate
1	$t_{TLB} + t_{H/MC} + t_{FDC} = 75\text{ns}$	$H_{TLB} \cdot H_C = 0,855$
2	$t_{TLB} + t_{H/MC} + t_{MP} = 250\text{ns}$	$H_{TLB} \cdot (1 - H_C) = 0,045$
3	$t_{TLB} + 2(t_{H/MC} + t_{FDC}) = 125\text{ns}$	$[(1-H_{TLB}) \cdot H_C] \cdot H_C = 0,09025$
4	$t_{TLB} + (t_{H/MC} + t_{FDC}) + (t_{H/MC} + t_{MP}) = 300\text{ns}$	$[(1-H_{TLB}) \cdot H_C] \cdot (1-H_C) = 0,00475$
5	$t_{TLB} + (t_{H/MC} + t_{MP}) + (t_{H/MC} + t_{FDC}) = 300\text{ns}$	$[(1-H_{TLB}) \cdot (1-H_C)] \cdot H_C = 0,00475$
6	$t_{TLB} + (t_{H/MC} + t_{MP}) + (t_{H/MC} + t_{MP}) = 475\text{ns}$	$[(1-H_{TLB}) \cdot (1-H_C)] \cdot (1-H_C) = 0,00025$

MMU: paginarea și memoria virtuală

7. Memoria virtuală în sistemele dotate cu memorie *cache*: Timpul mediu de acces la memorie:

Timpul mediu de acces la memorie va fi:

$$t_{med} = (75 \times 0,855) + (250 \times 0,045) + (125 \times 0,09025) + (300 \times 0,00475) + (300 \times 0,00475) + (475 \times 0,00025) = 89,9625 \text{ ns}$$

Dacă MMU utilizează 2 nivele de tabele pentru translatarea adresei virtuale (tabela de directori și tabele de pagini pe nivelul 2), vor exista și alte combinații care trebuie luate în considerare. De asemenea, calculul poate fi ușor adaptat pentru cazul în care există o suprapunere între accesele în TLB și accesele în *cache*.